

DPP Assignment 2

Simon Vinding Brodersen(rzn875)

December 3, 2025

1 Flattening with the New, More Efficient Rules

My code validates and is provided below:

```
def optimIII1Ker [m][n][q] (Sa: [m]u32, Da: [n]f32)
    (Sb: [m]u32, Db: [q]f32)
    (cs: [m]f32) (inds: [m]i64)
    : [m]f32 =
    let (Ba, flags'') = mkFlagArray Sa 0 (iota m)
    let flags' = map u32.i64 flags''
    let flags = map bool.u32 flags' :> [n]bool
    let flen_iot = reduce (+) 0 Sa |> i64.u32 |> iota :> [n]i64
    let iia1 = sgmScan (+) 0 flags (flags' :> [n]u32)
    let iia2 = map2(\i sgm -> (u32.i64 i) - Ba[i64.u32 sgm]) (flen_iot) iia1
    let (Bb, _) = mkFlagArray Sb false (replicate m true)
    let b_vals = map2 (\off ind -> Db[i64.u32 off + ind]) Bb inds
    let tmp1s = map2 (\a sgm -> (f32.sqrt a) * b_vals[i64.u32 sgm] + cs[i64.u32 sgm]) Da iia1
    let iot1s = iia2
    let iotfs = map f32.u32 iot1s
    let tmp2s = map2 (+) tmp1s iotfs
    let tmp_scan = sgmScan f32.max f32.lowest flags tmp2s
    let res = imap2 (iota m) Sa
        (\i s -> if s <= 0 then f32.lowest
            else if i == m-1
                then tmp_scan[n-1]
                else tmp_scan[i64.u32 Ba[i+1]-1]
        )
    in res
```

The runtime comparison between the standard classic implementation and the new optimized implementation can be seen below:

(m, q, p)	Classic (μs)	optimIII1 (μs)	Speedup
(10, 10 000 000, 100 000)	11212	7808	1.44
(100, 1 000 000, 10 000)	11192	7810	1.43
(1000, 100 000, 1000)	11250	7807	1.44
(10000, 10 000, 100)	11269	7866	1.45
(100000, 1000, 10)	11340	7908	1.43

Table 1: Performance comparison of classic and optimIII1 kernels with speedup.

This means that the new optimized implementation improves performance by around 1.44 across the board.

2 Flattening Rule for Scatter inside Map

Let Sx and Si denote the shape of the inner arrays of xss and iss same as S_{arr}^1 in the slides, let Dx denote the flat data of xss , let Di denote the flat data of iss and Dv be the flat data of vss . Then we would have the following flattening rules for the scatter inside a map:

```

let (Bx, _) = mkFlagArray Sx 0 (iota (length Sx))
let (Bi, flags') = mkFlagArray Si (iota (length Si))
let flags = map bool.u32 flags'
let ii1s = sgmScan (+) 0 flags flags'
let glob_idxxs = map2 (\sgm ind -> ind + Bx[sgm]) ii1s Di
in res = scatter Dx glob_idxxs Dv

```

$Bx[i]$ tells us the starting location of the inner list i in Dx . We then find the segment indices for the iss , such that we can add the offset, $Bx[sgm]$, to each of the inner lists of iss . This then gives us the final global index of each of the values. This then means we can directly scatter into Dx with the global indices and given each index corresponds to some value in vss , we don't have to change anything to Dv .

3 Flattening Rule for Histogram inside Map

Much the same as before, let Sx and Si the shape of the inner arrays of $histos$ and iss , same as S_{arr}^1 in the slides, let Dx denote the flat data of $histos$, let Di denote the flat data of iss and Dv be the flat data of vss .

Then we have the following flattening rules for the `reduce_by_index` inside a map:

```

let (Bx, _) = mkFlagArray Sx 0 (iota (length Sx))
let (Bi, flags') = mkFlagArray Si (iota (length Si))
let flags = map bool.u32 flags'
let ii1s = sgmScan (+) 0 flags flags'
let glob_idxxs = map2 (\sgm ind -> ind + Bx[sgm]) ii1s Di
in res = reduce_by_idx (⊙) 0⊙ Dx glob_idxxs Dv

```

The argument for correctness is analogous to that of the previous task.

4 Lifted version of partition2